

Otimização, Automação Procedural e Controle de Concorrência em Banco de Dados Relacional

Laboratório 02 / Parte 2 – Disciplina de Banco de Dados (2026/2) – Prof. Me. Moises Silva de Sousa

Amanda de Jesus Ferreira

Ciência da Computação / Engenharia de Software
Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP)
Brasília, Brasil

Rafael Tominaga

Ciência da Computação / Engenharia de Software
Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP)
Brasília, Brasil

Abstract—Este artigo relata a implementação prática de técnicas avançadas de engenharia de dados sobre o esquema relacional *bd_locadora*. Foram projetadas visões relacionais para abstração analítica e índices em árvore B-Tree para mitigação de varreduras sequenciais completas (*Full Table Scans*), comprovadas pela inspeção de planos de execução via comando *EXPLAIN*. Na camada procedural, implementaram-se gatilhos de sincronização de estados e auditoria com tabelas de log, além de *Stored Procedures* e funções para encapsulamento de regras de negócio. Por fim, realizaram-se experimentos controlados de concorrência em múltiplas sessões, avaliando bloqueios pessimistas (*FOR UPDATE*), indução de *deadlocks* e o comportamento do mecanismo de Controle de Concorrência Multiversão (MVCC) sob diferentes níveis de isolamento transacional no motor de armazenamento InnoDB.

Index Terms—Views, Índices B-Tree, *EXPLAIN*, Triggers, *Stored Procedures*, Concorrência, *Deadlock*, MVCC, MySQL InnoDB.

I. INTRODUÇÃO

À medida que as aplicações corporativas escalam em volume de transações e concorrência de acessos, a integridade e o tempo de resposta do Sistema de Gerenciamento de Banco de Dados (SGBD) tornam-se fatores críticos. Diferente de sistemas de arquivos convencionais, que carecem de garantias transacionais e métodos declarativos eficientes de recuperação, os SGBDs modernos disponibilizam componentes avançados para otimização de consultas e coordenação de transações simultâneas.

Neste contexto, o presente relatório documenta a evolução da base de dados *bd_locadora*, contemplando:

- 1) Abstração de relatórios recorrentes via Visões (*Views*);
- 2) Indexação seletiva com avaliação de planos de execução através da instrução *EXPLAIN*;
- 3) Automação de integridade reativa e auditoria irrefutável com *Triggers*;
- 4) Encapsulamento de fluxos operacionais via *Stored Procedures* e *Functions*;

- 5) Análise experimental de contenção de travas (*locking*), resolução de impasses mútuos (*deadlocks*) e isolamento via MVCC (*Multi-Version Concurrency Control*).

II. PARTE 1: VISÕES E OTIMIZAÇÃO COM ÍNDICES

A. Agregação e Abstração com Views

Visões relacionais foram materializadas para simplificar interfaces de consulta da locadora, ocultando a complexidade de junções e computações agregadas:

- *vw_locacoes_ativas*: Consolida contratos em curso através da junção de *locacao*, *cliente* e *veiculo*, filtrando apenas tuplas onde *data_real_devolucao* IS NULL.
- *vw_veiculos_disponiveis*: Expõe a frota apta para locação vinculada aos valores de diária da tabela *categoria*, filtrando *status* = 'Disponível'.
- *vw_faturamento_mensal*: Agrupa os montantes faturados por ano e mês da data de retirada (*YEAR*, *MONTH*) e filial de origem, aplicando a função de agregação *SUM(l.valor_total)*.

B. Análise do Plano de Execução (*EXPLAIN*) e Indexação

Avaliou-se o plano de execução de três consultas frequentes antes e após a criação de estruturas de indexação secundárias em árvore B-Tree.

1) *Consulta A (Busca por CPF)*: Filtro de busca pontual na tabela *cliente*.

- **Antes do Índice Secundário**: O plano exibe *type* = *const* utilizando a chave natural de unicidade prévia (*possible_keys*: *cpf*).
- **Índice Criado**:

```
CREATE INDEX idx_cliente_cpf ON cliente(cpf);
```
- **Depois do Índice**: O otimizador passa a reconhecer ambos os caminhos indexados (*possible_keys*: *cpf*, *idx_cliente_cpf*), mantendo a busca direta pontual em tempo logarítmico.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	cliente	NULL	const	cpf	cpf	44	const	1	100.00	NULL

Fig. 1. EXPLAIN da Consulta A antes da criação do índice secundário.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	cliente	NULL	const	cpf,idx_cliente_cpf	cpf	44	const	1	100.00	NULL

Fig. 2. EXPLAIN da Consulta A após o índice idx_cliente_cpf.

2) *Consulta B (Filtro por Intervalo de Retirada)*: Busca delimitada por faixa temporal na tabela locacao.

- **Antes do Índice:** O motor executou varredura completa na tabela com `type = ALL` (*Full Table Scan*), inspecionando todas as linhas sem chaves disponíveis (`key = NULL`).
- **Índice Criado:**

```
CREATE INDEX idx_locacao_data_retirada ON locacao(data_retirada);
```
- **Depois do Índice:** O otimizador migrou para varredura em faixa (`type = range`), utilizando a chave `idx_locacao_data_retirada` com a condição *Using index condition*.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	locacao	NULL	ALL	NULL	NULL	NULL	NULL	5	20.00	Using where

Fig. 3. EXPLAIN da Consulta B antes da indexação (*Full Table Scan*, `type = ALL`).

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	locacao	NULL	range	idx_locacao_data_retirada	idx_locacao_data_retirada	5	NULL	2	100.00	Using index condition

Fig. 4. EXPLAIN da Consulta B com `type = range` após o índice.

3) *Consulta C (Filtro de Veículo por Categoria)*: Pesquisa de frota filtrada pela chave de categoria.

- **Antes do Índice Dedicado:** O otimizador empregava o índice da chave estrangeira original (`key = fk_veiculo_categoria`).
- **Índice Criado:**

```
CREATE INDEX idx_veiculo_categoria ON veiculo(id_categoria);
```
- **Depois do Índice:** A consulta passou a utilizar explicitamente o índice secundário dedicado `idx_veiculo_categoria` com `type = ref`.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	veiculo	NULL	ref	fk_veiculo_categoria	fk_veiculo_categoria	4	const	2	100.00	NULL

Fig. 5. EXPLAIN da Consulta C com chave estrangeira preexistente.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	veiculo	NULL	ref	idx_veiculo_categoria	idx_veiculo_categoria	4	const	2	100.00	NULL

Fig. 6. EXPLAIN da Consulta C após a criação do índice `idx_veiculo_categoria`.

C. Pergunta de Reflexão: Degradação por Excesso de Índices

A presença excessiva de índices secundários penaliza sensivelmente o desempenho do SGBD em cargas de trabalho intensivas em escrita (ambientes OLTP com elevadas taxas de INSERT, UPDATE e DELETE). Os principais fatores compreendem:

- 1) **Custo de Manutenção da Estrutura B-Tree:** Para cada nova tupla inserida ou removida, o SGBD precisa reequilibrar não apenas a tabela física primária, mas todas as árvores de índices secundários correspondentes, provocando divisões e junções de páginas (*page splits*).
- 2) **Amplificação de I/O e Gravação de Logs:** A escrita repetida de nós exige gravação contínua nos arquivos de log transacional (*Write-Ahead Logging / Redo Log*), saturando o subsistema de armazenamento.
- 3) **Competição no Buffer Pool:** Índices consomem páginas de memória RAM no *Buffer Pool*. Se o volume de índices exceder a memória dedicada, dados operacionais frequentes são descartados prematuramente para o disco, reduzindo a taxa de acerto de cache (*cache hit ratio*).
- 4) **Sobrecarga no Otimizador:** Um número elevado de índices aumenta o custo computacional gasto pelo otimizador de consultas para avaliar combinações de planos durante o planejamento da execução.

III. PARTE 2: TRIGGERS, PROCEDURES E FUNCTIONS

A. Automação e Auditoria com Triggers

Desenvolveram-se rotinas reativas disparadas automaticamente em eventos de manipulação:

- **Atualização de Status de Veículo:** O gatilho `trg_locacao_insert_veiculo` (AFTER INSERT) comuta o status do veículo associado para `'locado'`. Complementarmente, o gatilho `trg_locacao_update_veiculo` monitora a transição de devolução (`OLD.data_real_devolucao IS NULL` e `NEW.data_real_devolucao IS NOT NULL`), restaurando o estado para `'Disponível'`.
- **Auditoria de Contratos (`log_locacao`):** Criou-se uma tabela dedicada de auditoria com rastreamento temporal (`NOW()`) e de autoria (`CURRENT_USER()`). O gatilho `trg_locacao_audit` intercepta alterações na tabela `locacao` e grava o histórico sempre que ocorrer divergência entre valores monetários ou transições entre status lógicos.

B. Lógica Procedural com Procedures e Functions

Regras de cálculo e integridade foram centralizadas no banco:

- **Procedure `sp_abrir_locacao`:** Encapsula a abertura atômica de reservas recebendo os identificadores de cliente, veículo, filial e a quantidade de dias. Caso o veículo não esteja disponível, dispara uma interrupção formal via `SIGNAL SQLSTATE '45000'`. Estando

apto, recupera o valor da diária, computa o montante contratual total e efetua a persistência.

- **Function `fn_calcula_multa`:** Função determinística que avalia a diferença temporal em dias entre a devolução prevista e a devolução real (`DATEDIFF`). Se houver atraso consumado, calcula o produto dos dias excedentes pela taxa diária de multa estipulada; caso contrário, retorna zero.

IV. PARTE 3: CONCORRÊNCIA, TRANSAÇÕES E MVCC

A. Bloqueio Explícito com `FOR UPDATE`

Simulou-se a contenção de acesso ao mesmo recurso físico por duas transações simultâneas.

- A **Sessão A** iniciou transação e adquiriu trava exclusiva de linha (*Row-Level Exclusive Lock*) via comando `SELECT ... FOR UPDATE` sobre o veículo 1.
- A **Sessão B**, ao submeter a mesma instrução concorrentemente, entrou em estado de suspensão (*Lock Wait*), aguardando a desocupação do recurso.
- Ao ser emitido o `COMMIT` na Sessão A, a trava foi liberada e a Sessão B completou sua execução instantaneamente, demonstrando o controle de isolamento pessimista contra reservas concorrentes.

B. Indução e Detecção de Deadlock

Forçou-se uma dependência circular de travamento cruzado através de operações transacionais alternadas entre duas conexões independentes.

TABLE I
CRONOGRAMA DE EXECUÇÃO PARA INDUÇÃO DE DEADLOCK

Passo	Sessão A (Conexão 10)	Sessão B (Conexão 12)
1	<code>START TRANSACTION;</code>	–
2	<code>UPDATE cliente SET ... (id=1)</code>	–
3	–	<code>START TRANSACTION;</code>
4	–	<code>UPDATE veiculo SET ... (id=1)</code>
5	<code>UPDATE veiculo SET ... (id=1)</code> (<i>Suspensa aguardando Sessão B</i>)	–
6	–	<code>UPDATE cliente SET ... (id=1)</code> (DEADLOCK DETECTADO!)

No Passo 6, a Sessão B demandou a trava do registro retido pela Sessão A, configurando um ciclo fechado no Grafo de Espera (*Wait-For Graph*). O motor InnoDB identificou o impasse e abortou a transação da Sessão B emitindo o erro oficial de deadlock:

6: 09:11:04: START TRANSACTION	0 rows affected	0.000 sec
7: 09:11:04: UPDATE veiculo SET modelo = 'Tesla B' WHERE id_veiculo = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
8: 09:11:06: UPDATE cliente SET nome = 'Tadeu B' WHERE id_cliente = 1	Error Code: 1213. Deadlock found when trying to get lock; try restarting transaction	0.010 sec

Fig. 7. Evidência do erro 1213 de Deadlock capturado no MySQL Workbench.

1) **Prevenção de Deadlocks:** A prevenção arquitetural contra impasses mútuos fundamenta-se na imposição de uma **Ordem Global e Padronizada de Acesso aos Recursos**. Se todas as transações da aplicação forem programadas para acessar e travar as tabelas invariavelmente na mesma sequência hierárquica (por exemplo, sempre manipular `cliente` antes de manipular `veiculo`), o grafo direcionado de dependência permanecerá acíclico, eliminando a ocorrência de bloqueios mútuos.

C. Níveis de Isolamento e MVCC

O comportamento do versionamento de dados sob controle multiversão (MVCC) foi confrontado em dois patamares de consistência:

- **Prevenção de Leitura Suja (*Dirty Read*):** Uma alteração efetuada na Sessão A sem `COMMIT` não se tornou visível para a Sessão B sob o nível `READ COMMITTED`. O motor InnoDB utilizou os ponteiros de reversão gravados no *undo log* para reconstruir o *snapshot* consistente anterior, garantindo isolamento sem travar leituras.
- **Leitura Não Repetível vs. Repetível:** Sob `READ COMMITTED`, novas consultas após a confirmação da Sessão A revelaram o dado atualizado. Já sob `REPEATABLE READ`, o MySQL congelou a visão do catálogo no momento do primeiro `SELECT` da Sessão B, retornando valores consistentes durante todo o escopo de vida da transação.

V. EXPERIÊNCIA E DIVISÃO DE TRABALHO DOS INTEGRANTES

Em cumprimento às diretrizes acadêmicas da disciplina, a distribuição das etapas entre a dupla foi estabelecida da seguinte forma:

- **Amanda de Jesus Ferreira:** Responsável pelo projeto e implementação das três visões relacionais (*Views*), formulação das instruções de análise do `EXPLAIN`, análise teórica da perda de desempenho associada à proliferação de índices secundários e condução da Sessão A nos ensaios práticos de concorrência e transações.
- **Rafael Tominaga:** Atuou na codificação e teste da arquitetura procedural (gatilhos de sincronização, tabela/gatilho de auditoria, *Stored Procedure e Function*), operacionalização da Sessão B na simulação do bloqueio `FOR UPDATE` e na indução deliberada do Deadlock (Erro 1213), além da diagramação do relatório no padrão IEEE no Overleaf.

VI. CONCLUSÃO

Os experimentos evidenciaram que a eficiência de um SGBD corporativo transcende o mero armazenamento de tabelas relacionais. A indexação adequada minimizou os custos de varredura física, enquanto rotinas procedurais integradas e o conhecimento dos protocolos de concorrência e travamento pessimista garantiram a atomicidade dos negócios e a integridade da locadora frente a acessos altamente concorrentes.

REFERENCES

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.
- [3] Oracle Corporation, *MySQL 8.0 Reference Manual*, Oracle, 2024.